Mohammad Ali Jinnah University
Karachi, Pakistan

Drive Ease - Final OOP Lab Project Report

Subscription-Based Car Rental & Maintenance Hub

Submitted By:

NAME: FARHAN HAROON ID: FA25-BSAI-0060

NAME: BISSAM ANSARI ID: FA25-BSAI-0076

1. Project Overview

DriveEase is a console-based car rental and maintenance management system built in C++. It allows two types of users — Admin and Customer — to interact with a shared fleet of vehicles. Customers can browse available vehicles, make bookings, return or cancel them, and manage their subscriptions. Admins can service vehicles, view reports, and upgrade customer plans.

The project was developed as part of the OOP Lab to demonstrate the four core pillars of object-oriented programming: Encapsulation, Abstraction, Inheritance, and Polymorphism, along with Composition as a structural technique.

2. OOP Pillars Demonstrated

Encapsulation

All class attributes are declared private. Access is strictly controlled through validated getters and setters. For example, MaintenanceRecord rejects negative costs and empty service dates at the setter level, preventing invalid state from entering the object.

Abstraction

Two abstract base classes define contracts for the system. RentalEntity mandates that any rentable object exposes an ID, name, daily rate, and rental fee calculation. IReport mandates that any reporting entity provides both a full and a date-filtered report. Concrete classes must fulfill these contracts to compile.

Inheritance

The Vehicle class extends RentalEntity and is itself abstract. Three concrete vehicle types — Car, SUV, and LuxuryCar — inherit from it, each adding their own attributes. Similarly, SubscriptionPlan is an abstract base from which BasicPlan, StandardPlan, and PremiumPlan are derived. RentalHub inherits from IReport.

Polymorphism

Runtime polymorphism is achieved through virtual method overrides. Each vehicle type overrides calculateRentalFee() differently: Car applies a 5% discount for bookings longer than 3 days, SUV adds a flat Rs.500 premium, and LuxuryCar adds Rs.800 per day if a chauffeur is included. Compile-time polymorphism is shown through overloaded generateReport() — one version prints a full hub report, another filters by date.

Composition

Three composition relationships exist. Vehicle has-a MaintenanceRecord, storing the full service history of each vehicle. Customer has-a vector of Booking objects, each holding rental details and status. RentalHub has-a vector of Customer objects and a fleet of Vehicle pointers.

3. Class Structure

The table below summarises the key classes and their roles in the system.

Class	Role
RentalEntity	Abstract base — defines the rental contract for all rentable entities
IReport	Interface — mandates full and date-filtered report generation
Vehicle	Abstract base for all vehicle types; holds fleet-common attributes
Car / SUV / LuxuryCar	Concrete vehicle types with custom rental fee calculations
SubscriptionPlan	Abstract base for all subscription tiers
BasicPlan / StandardPlan / PremiumPlan	Concrete plans with different pricing, rental limits, and discounts
MaintenanceRecord	Composition class — stores service history inside each Vehicle
Booking	Composition class — holds rental details inside each Customer
Customer	Stores customer profile, subscription, and booking history
RentalHub	Central hub; manages the fleet and customer list; implements IReport

4. System Features

Admin Role

- Login with username and password (admin / admin123)
- Service a vehicle and log cost, date, and notes to its MaintenanceRecord
- View all registered customers
- Generate a full hub report or filter bookings by date
- Upgrade a customer's subscription plan

Customer Role

- Register a new account with a chosen subscription plan
- Login using a personal Customer ID
- Browse the available fleet with live availability status
- Rent a vehicle (subject to plan rental limits and availability)
- Return a rented vehicle
- Cancel an active booking

5. Error Handling

The project uses C++ exception handling throughout. All invalid input — negative costs, bad dates, duplicate IDs, unavailable vehicles, exceeded plan quotas — throws a std::invalid_argument or std::runtime_error that is caught at the session loop level. This ensures the program never crashes on bad input; it prints a descriptive error and resumes.

6. Pre-loaded Test Data

The system seeds six vehicles on startup: two Cars (Toyota Corolla, Honda Civic), two SUVs (Toyota Fortuner, Kia Sportage), and two Luxury Cars (Mercedes E-Class with chauffeur, BMW 7 Series self-drive). Four customers are pre-registered across all three subscription tiers. This allows immediate testing of all system features without manual registration.

Uml Diagram Representation: